

# Token Expand-Merge: Training-Free Token Compression for Vision-Language-Action Models

Yifan Ye<sup>1,\*</sup>, Jiaqi Ma<sup>2,\*</sup>, Jun Cen<sup>3</sup>, Zhihe Lu<sup>1,†</sup>

**Abstract**—Vision-Language-Action (VLA) models pretrained on large-scale multimodal datasets have emerged as powerful foundations for robotic perception and control. However, their massive scale, often billions of parameters, poses significant challenges for real-time deployment, as inference becomes computationally expensive and latency-sensitive in dynamic environments. To address this, we propose Token Expand-and-Merge-VLA (TEAM-VLA), a training-free token compression framework that accelerates VLA inference while preserving task performance. TEAM-VLA introduces a dynamic token expansion mechanism that identifies and samples additional informative tokens in the spatial vicinity of attention-highlighted regions, enhancing contextual completeness. These expanded tokens are then selectively merged in deeper layers under action-aware guidance, effectively reducing redundancy while maintaining semantic coherence. By coupling expansion and merging within a single feed-forward pass, TEAM-VLA achieves a balanced trade-off between efficiency and effectiveness, without any retraining or parameter updates. Extensive experiments on LIBERO benchmark demonstrate that TEAM-VLA consistently improves inference speed while maintaining or even surpassing the task success rate of full VLA models. The code is public available on <https://github.com/Jasper-aaa/TEAM-VLA>

## I. INTRODUCTION

Vision-Language-Action (VLA) models have recently demonstrated strong performance across both real-world and simulated robotic control tasks, largely owing to the powerful representation capabilities of Large Vision-Language Models (LVLMs) [1], [2], [3]. While LVLM backbones offer rich multimodal understanding and robust generalization, they also impose substantial computational and memory overhead. This significantly limits their practicality in high-frequency, low-latency control scenarios such as real-time manipulation, closed-loop feedback policies, and on-device robotics [4], [5], [6]. As a result, improving the computational efficiency of VLA models, without sacrificing their action reasoning ability, remains a critical yet underexplored challenge for scalable deployment.

Existing works often mitigate this computational burden through token-level pruning, which accelerates inference by reducing the number of visual tokens processed by the VLA backbone [7], [8], [9], [10], [11]. While effective, these approaches typically depend on trainable query

Yifan Ye and Zhihe Lu are with <sup>1</sup>College of Science and Engineering, Hamad Bin Khalifa University, Education City, Doha 24404, Qatar

Jiaqi Ma is with <sup>2</sup>Mohamed bin Zayed University of Artificial Intelligence, Abu Dhabi 23201, UAE

Jun Cen is with <sup>3</sup>College of Computer Science and Technology, Zhejiang University, Hangzhou 310058, China

\* Equal contribution.

† Corresponding Authors, [zlu@hbku.edu.qa](mailto:zlu@hbku.edu.qa),

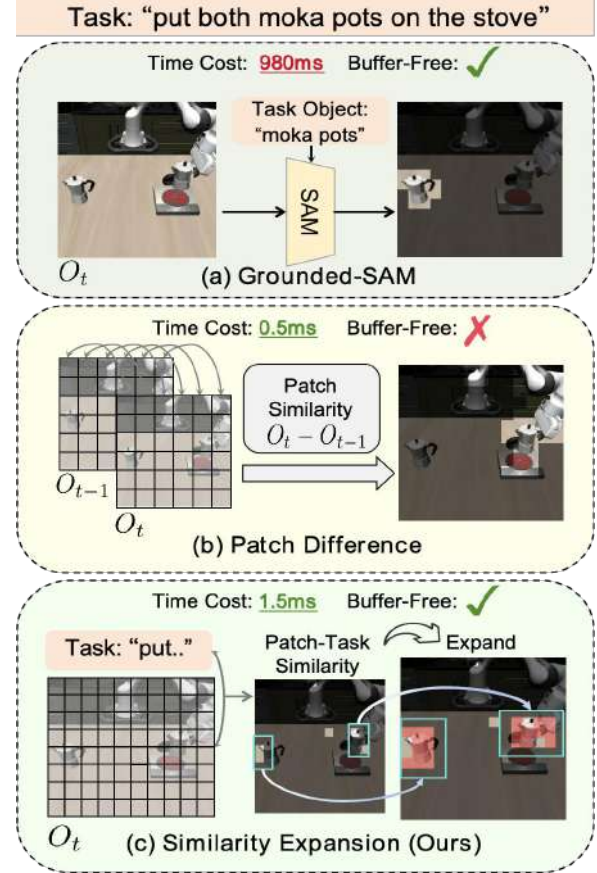


Fig. 1: A visual comparison of three foreground extraction strategies shows that our similarity-driven expansion achieves the most coherent foreground regions while maintaining superior efficiency and zero-buffer overhead.

mechanisms [9] or cross-frame temporal cues [12], [13], [8] to identify and retain salient foreground tokens. Such designs often require either additional training or access to previous observations  $O_{t-1}$  during inference, which increases system complexity, introduces memory overhead, and may degrade robustness when temporal continuity is unreliable (e.g., abrupt viewpoint changes or partial observability). Consequently, a training-free, and temporally independent token compression strategy remains highly desirable for practical VLA deployment.

In this paper, we propose a training-free, observation-only token compression method, named **Token Expanding And Merging for Vision-Language-Action models (TEAM-VLA)**. TEAM-VLA identifies foreground-relevant visual to-

kens purely from the current frame, without relying on trainable queries or temporal buffering. Prior works [9], [12], [11], [14] suggest that the similarity between projected visual tokens and language embeddings can serve as an indicator of object relevance. However, as illustrated in Figure 1, the maximum similarity over all task language tokens yields extremely sparse responses. This sparsity arises because task descriptions contain many non-object terms (e.g., “put”, “the”, “on”), each mapping to isolated and semantically irrelevant pixels, while even object-centric phrases (e.g., “both moka pots”) correspond to only a few token-level anchors. This motivates the need to reconstruct the full spatial extent of the underlying objects from these sparse cues. To address this, we introduce a Token Expanding mechanism that propagates high-similarity seeds into coherent spatial regions. A smoothing convolutional scan is applied to selectively expand linguistically meaningful areas, while noisy or weak-response regions are supplemented through a controlled random expansion to preserve potential foreground candidates. To maintain overall structural completeness, we further enrich the expanded regions with a small fraction of randomly sampled contextual tokens.

Beyond expansion, we observe that action–text interactions at intermediate layers of the VLA backbone reveal additional visual tokens that encode task-relevant motion cues or spatial structure. Although these tokens may exhibit weaker object relevance, discarding them risks losing important functional information. Therefore, TEAM-VLA retains the top- $M$  action–text-responsive tokens and applies a soft bipartite merging mechanism to compress the remaining tokens into semantically aligned groups. This ensures that informative yet subtle cues remain preserved in compact form. Extensive experiments on the LIBERO benchmark [15] demonstrate that TEAM-VLA achieves an excellent success–latency trade-off, substantially accelerating inference while maintaining strong execution performance.

In summary, our contributions are threefold.

- We introduce TEAM-VLA, a fully training-free and observation-only token compression framework for Vision–Language–Action models, requiring no additional supervision, past-buffering, or model retraining.
- We develop a foreground-aware token compression pipeline that combines (i) a fast similarity-driven *Token Expanding* module to reconstruct dense foreground regions from sparse vision–language cues, and (ii) an action-guided soft-bipartite *Token Merging* mechanism that compresses deeper-layer tokens while preserving essential semantic structure.
- Extensive experiments on the LIBERO benchmark demonstrate that TEAM-VLA achieves an excellent trade-off between token reduction and action success rate, offering a practical acceleration strategy for VLA deployment.

## II. RELATED WORK

### A. Vision–Language–Action Models

Vision–Language–Action (VLA) models extend the capabilities of large vision–language models (VLMs) [16], [17], [18] by integrating a visual–language encoder with an action-generation head [4], [2], [19], [20]. This unified architecture enables robots to perceive, interpret, and act within diverse environments, yielding strong performance in both simulation and real-world deployments [21], [22]. Early efforts in this direction include the RT-series models [23], [24], which combine pretrained VLMs with large-scale robot demonstrations, showcasing the potential of VLA systems to achieve robust semantic understanding and reliable control. Subsequent works have further advanced this paradigm:  $\pi 0$  and OpenVLA [1] demonstrate remarkable zero-shot generalization by training on extensive real-world datasets, while OpenVLA-OFT [3] introduces techniques such as action chunking and parallel decoding to significantly boost execution accuracy—achieving over 95% on the LIBERO benchmark [15]. These developments highlight the growing maturity of VLA architectures and underline the importance of efficient, scalable methods for real-world deployment.

### B. Token Compression for VLA

Large VLMs (LVLMs) provide powerful semantic grounding for Vision–Language–Action (VLA) systems, yet their heavy Transformer backbones introduce substantial computational overhead when processing long visual token sequences [25], [6]. As a result, pruning redundant visual tokens has emerged as an effective strategy to improve efficiency in multimodal LVLMs [11], [26], [22]. The core challenge in token pruning is to distinguish informative tokens from unimportant ones [7], [27], [14], [28]. Prior work frequently leverages text–image cross-attention maps to determine which patch tokens to retain [11], [27], [29], an approach that works well in general LVLMs because they contain rich and diverse language embeddings. However, in robot manipulation, language tokens are typically limited to the task instruction and thus provide sparse guidance. This sparsity causes cross-attention–based pruning to degrade, often requiring additional learnable modules or adapters to compensate for the insufficient attention signals [9], [6].

Beyond static cross-attention pruning, several studies exploit the streaming nature of robotic perception by identifying regions of interest through inter-frame changes [12], [8], [13], [7]. Methods such as VLA-Cache [13] and SpecPrune-VLA [12] adopt a two-frame comparison strategy to select dynamic or foreground-relevant tokens, effectively preserving motion cues. However, these approaches rely on temporal information from previous frames, making them dependent on buffering and assumptions about frame-to-frame consistency. In contrast, we propose a training-free, observation-only token pruning and merging framework that operates solely on the current frame. By combining first-layer pruning with deep-layer merging, our TEAM-VLA achieves competitive performance without requiring temporal cues, additional training, or reliance on cross-frame priors.

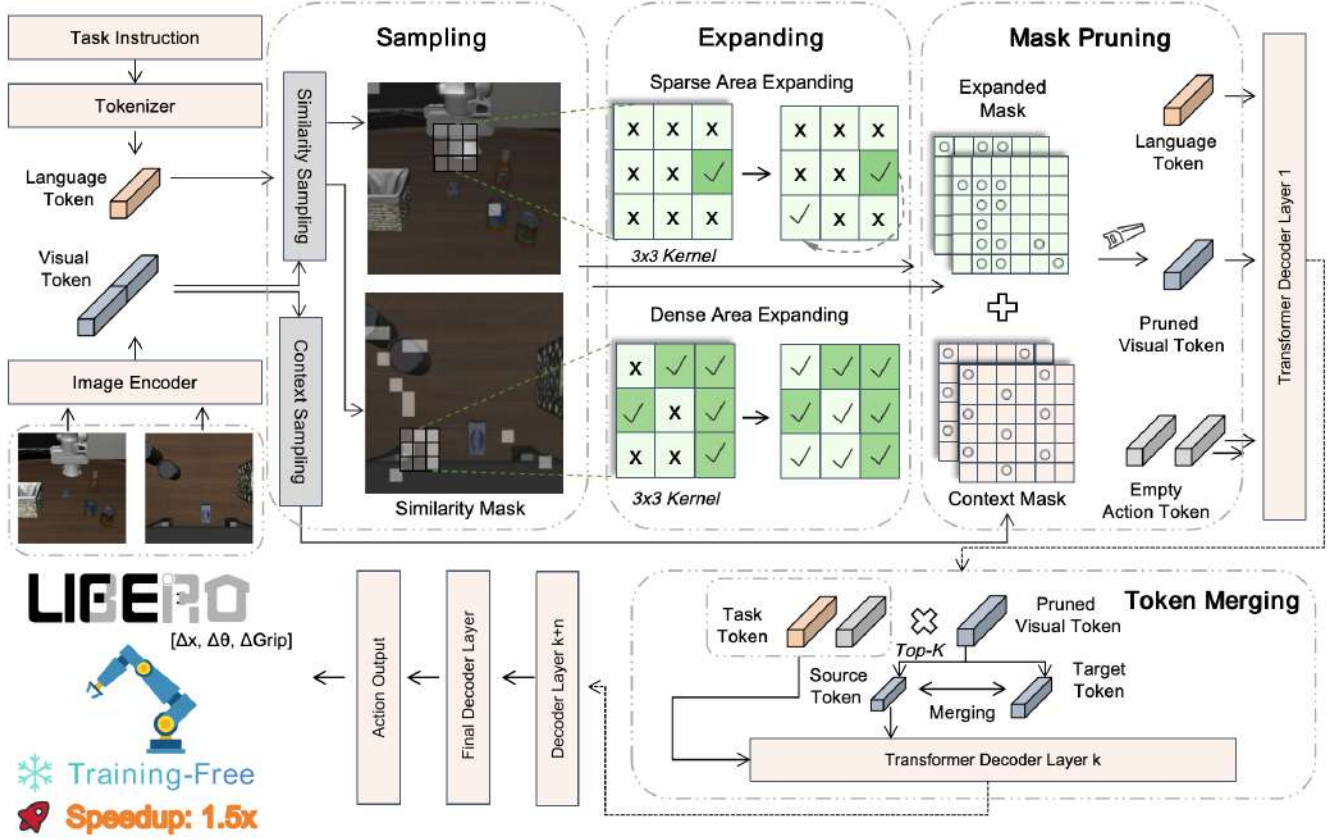


Fig. 2: The overall pipeline of TEAM consists of two stages. First, before tokens enter the language backbone, we perform token pruning. This begins with two complementary sampling steps: (1) similarity-based sampling and (2) context sampling, followed by a spatial expansion module that densifies the sparse similarity map. The expanded mask is then used to remove redundant visual tokens while preserving task-relevant regions. Second, at a middle layer of the backbone, we introduce an action-guided soft bipartite matching module that merges tokens through weighted averaging, effectively compressing deep representations while retaining essential semantic and action-related information.

### III. PRELIMINARIES

#### A. Vision-Language-Action Models

A typical vision-language action model  $\pi_\theta$  generally consists of a sensory encoder, a language model backbone, and an action head [4], [21]. The sensory encoder typically includes an image encoder, a language encoder, and a robot proprioception (state) encoder. These encoders transform the corresponding observation images  $o_t$ , task instructions  $l$ , and robot states  $s$  into embeddings  $E_{img}, E_{lang}, E_{state}$ , which are then passed to the language backbone and the action head for generating action  $a$ . Formally we defined a vision language action model conditioned on single observation as:

$$a_t = \pi_\theta(a_t | o_t, l, s) \quad (1)$$

#### B. Token Pruning

Given a sequence of tokens  $T = \{t_1, t_2, t_3, \dots, t_L\}$  as input to a language model (Transformer backbone), the token pruning technique removes a portion of tokens and retains a subset  $T_{pruned} \subset T$ . In both VLMs and VLAs, the self-attention mechanism introduces an  $O(n^2)$  computational complexity, meaning that the overall computation increases quadratically

with the number of tokens. Therefore, token pruning can significantly reduce the computational time required by the model without modifying its parameters.

### IV. METHODS

In this section, we present the detailed architecture of our proposed TEAM-VLA. We first describe the overall architecture of TEAM-VLA, followed by the proposed context sampling and highlight expanding modules. Finally, we introduce the action-guided merging strategy.

#### A. Overall Framework

As illustrated in Figure 2, TEAM-VLA accelerates inference through a dual strategy of early pruning and mid-layer merging. After the sensory encoder extracts visual features, we first perform context pruning and similarity expansion to discard redundant background tokens before they enter the LLM backbone. This removes a large portion of spatially uninformative tokens at the earliest stage. Then, at an intermediate backbone layer, we apply our action-guided soft bipartite merging to compress representations by preserving the top- $M$  tokens most relevant to the action token. Unlike prior training-free methods [12], [10] that prune only in



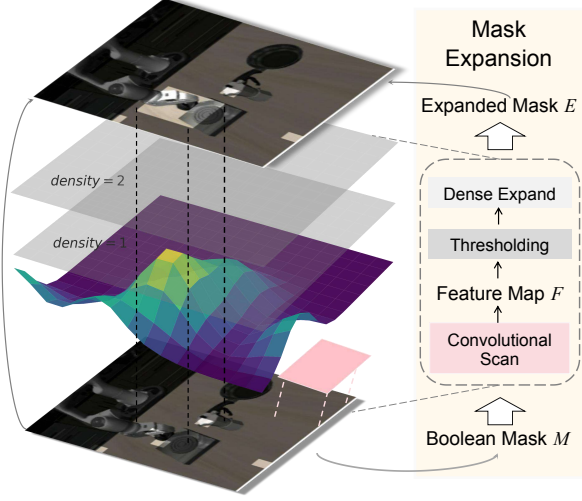


Fig. 3: We visualize the density distribution of the feature map  $F$  to illustrate how attended regions aggregate spatially. On the binary mask, we apply a convolutional operation to obtain a density feature map  $F$ . We then expand the regions based on their density values, where the areas with the highest density are fully expanded to cover their corresponding spatial neighborhoods.

deeper layers (where token counts are already low), our dual-reduction design eliminates redundancy both before and within the backbone, enabling substantially greater speed-up without sacrificing task performance.

### B. Token Pruning

1) *Motivation*: Token pruning requires efficiently retaining tokens that likely correspond to foreground regions of interest. A direct solution is to apply a segmentation model (Figure 1), but powerful foundation models such as Grounded-SAM [30], [31] are slow and often fail to localize objects solely from task instructions—making them impractical for real-time control. Other approaches [13], [12] detect dynamic tokens via temporal differences, but they require buffering previous frames and inject task-specific priors, limiting generality. Similarity- or cross-attention-based methods [27], [32] are efficient, yet in a training-free setting they often produce overly sparse matches in the first layer (white activation spots in Figure 1) due to weak text–image alignment [9], [12]. These limitations motivate us to develop a fast, training-free mechanism for identifying potential foreground tokens directly from the current frame, before feeding them into the language backbone.

2) *Similarity Sampling and Token Expanding*: The goal of token expanding is to enlarge the sparse regions of task-related tokens. To achieve this, we first identify task-related sparse tokens by computing the cosine similarity between image tokens and language tokens. Formally, given a language embedding  $E_{lang}$  and image patch embedding  $E_{img}$ , we compute the most relevant image token for each language token using the following cosine similarity equation.

$$\text{Argmax}_i \left( \frac{E_{lang} \cdot E_{img,i}^\top}{\|E_{lang}\|_2 \|E_{img,i}\|_2} \right). \quad (2)$$

Among them, we identify the patch token with the highest similarity  $\text{Argmax}_i$  for each language token and treat these tokens as foreground anchors in the image. After identifying the most related image token for each language token, we binarize the resulting relevance scores to obtain a Boolean mask  $M \in \{0, 1\}^{p \times p}$ , where  $p = H(W)/s$  denotes the number of patches along the height and width and  $s$  is the patch size (with  $H$  and  $W$  the input image height and width). To estimate the local density of attended regions, we apply a convolution operator  $F = \text{Conv}(M; K)$ , where  $K \in \mathbb{R}^{k \times k}$  is a kernel with all entries equal to 1 and zero padding is used. The resulting density feature map  $F \in \mathbb{R}^{p \times p}$  records, for each spatial position, the number of related patches within its  $k \times k$  neighborhood (shown in Figure 3). We then update the mask via two types of regional expansion. For positions in dense neighborhoods, defined as

$$\mathcal{D} = \{(i, j) \mid F_{ij} > \tau\}, \quad (3)$$

we assign all originally unrelated locations within the corresponding neighborhoods to 1 (related), effectively performing a deterministic dilation. For sparse neighborhoods,

$$\mathcal{S} = \{(i, j) \mid 0 < F_{ij} < \tau\}, \quad (4)$$

we randomly flip one unrelated position in the local neighborhood of each  $(i, j) \in \mathcal{S}$  from 0 (unrelated) to 1 (related), ensuring minimal but non-fragmented spatial coverage. As illustrated in Figure 3, relying solely on the sparse similarity regions (boolean mask) is insufficient to recover the full extent of the object. In contrast, our method preserves the complete task-relevant region by expanding these sparse cues into a coherent foreground representation (white plus red areas). Even with multiple image inputs, by leveraging the convolution kernel’s ability to process batch inputs (*i.e.* wrist-view and agent-view), we effectively reduce the foreground identification process to just 1–2 ms.

3) *Context Sampling*: After identifying the foreground tokens, we randomly sample a small subset of background tokens as complementary context. To achieve this efficiently, we adopt a simple interval-based sampling strategy over the entire token sequence, controlled by a parameter  $u \in [0, 1]$ . This ensures that a lightweight portion of scene structure is preserved, allowing the model to maintain spatial awareness while keeping redundancy minimal.

### C. Token Merging

1) *Motivation*: In VLAs, token merging has been largely overlooked [4], despite being widely adopted in VLMs and transformer-based vision models [29], [14], [28]. We argue that while early-layer pruning effectively removes redundant spatial information and sharpens focus on the foreground, the subsequent stages should prioritize preserving this extracted foreground information—even when further reducing token count [33]. To mitigate information loss during this process, we introduce an action-guided token merging strategy that selectively consolidates tokens based on their relevance to the action semantics.

2) *Task-Guided Bipartite Merging*: Token merging typically divides the image token set  $IT$  into a source set  $S$  and a target set  $T$ , where  $|S| + |T| = |IT|$  and  $S \cap T = \emptyset$ . In our method, we leverage both text and action tokens to identify the top- $M$  most relevant image tokens as the source set  $S$ . This design ensures that, unlike random or sequential selection strategies, the merging process preserves the most action-critical visual information. Following the same similarity computation as in Equation 2, we extract the top- $M$  tokens to form  $S$ , while the remaining tokens constitute  $T$ . We then perform source-target merging via a soft bipartite matching scheme. Concretely, we compute a similarity matrix between  $S$  and  $T$ , which determines how each target token contributes to its closest source token during the weighted merging process. This preserves semantic structure while achieving efficient token reduction.

$$\mathbf{Sim} = \frac{\text{RMSNorm}(\mathbf{S}) \times \text{RMSNorm}(\mathbf{T})^\top}{\sqrt{d}}, \quad (5)$$

where RMSNorm denotes Root Mean Square Layer Normalization, and  $d$  represents the hidden dimension of the language backbone. For each target token, a probability distribution over all source tokens is obtained by applying a softmax along the source dimension:

$$\begin{aligned} \mathbf{W} &= \text{softmax}(\mathbf{Sim}, \text{dim} = -1), \\ W_{ij} &= \frac{\exp(\text{Sim}_{ij})}{\sum_{k=1}^{N_{\text{Sim}}} \exp(\text{Sim}_{ik})}. \end{aligned} \quad (6)$$

This defines a *soft matching matrix*  $\mathbf{W} \in \mathbb{R}^{N_T \times N_S}$ . The matched target representations are aggregated back to each source position:

$$\mathbf{A} = \mathbf{W}^\top \mathbf{T}, \quad A_j = \sum_{i=1}^{N_T} W_{ij} T_{i,i}, \quad (7)$$

resulting in  $\mathbf{A} \in \mathbb{R}^{N_S \times d}$ . We compute the total soft-matching weight received by each source token:

$$\mathbf{s} = \mathbf{W}^\top \mathbf{1}_{N_T}, \quad s_j = \sum_{i=1}^{N_T} W_{ij}. \quad (8)$$

Each source token is updated by adding the aggregated target features and normalized by its total soft weight:

$$\mathbf{S}' = \frac{\mathbf{S} + \mathbf{A}}{1 + |\mathbf{S}|}, \quad (9)$$

where the division is element-wise. Note that each target token is merged into its most similar source token, ensuring that semantically related information is preserved and preventing major information loss even under an aggressive token-reduction regime.

## V. EXPERIMENT

### A. Benchmark

In our experiments, we use the LIBERO benchmark [15], which comprises four task subsets: LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, and LIBERO-Long. LIBERO-Spatial evaluates spatial reasoning by requiring the robot to place a bowl relative to its current

position. LIBERO-Object assesses object recognition and manipulation across diverse items. LIBERO-Goal examines procedural generalization by varying task goals while keeping the object set fixed. Finally, LIBERO-Long includes ten long-horizon tasks that test the robot's ability to execute extended, temporally complex operations.

### B. Experiment Setup

1) *Compared Methods*: We compare TEAM-VLA with several state-of-the-art token-level efficiency methods designed for VLA models built on large VLM backbones. Specifically, we evaluate against VLA-Cache [13], EfficientVLA [10], SparseVLM [11], and SpecPrune-VLA [12]. VLA-Cache caches static visual tokens to reduce redundant computation; EfficientVLA prunes visual tokens to accelerate action decoding; and SpecPrune-VLA removes tokens deemed irrelevant under global-local attention.

2) *Evaluation Metrics*: We adopt three widely used evaluation metrics [9], [10], [27], [8] to systematically assess our proposed TEAM-VLA: Success Rate (SR), FLOPs, and CUDA latency. We use success rate to evaluate task performance, FLOPs to measure theoretical computation, and CUDA latency to capture actual GPU runtime. We report the average CUDA latency over the ten tasks within each task suite.

3) *Implementation Details*: Following previous token acceleration methods, we implement our proposed TEAM-VLA on OpenVLA-OFT [3]. Specifically, we utilize the codebase of LightVLA [9]. We re-ran OpenVLA-OFT and other baselines on our platform and observed that the success rate of OpenVLA-OFT was lower than that reported in previous studies, even after averaging across multiple random seeds [9]. However, for a fair and intuitive comparison, we report all baseline performances from SpecPrune-VLA [12] and other baseline [10], [11], which were conducted on similar devices with same CUDA latency and higher performance than our implementation. In the parameter settings, due to varying task difficulties, we apply different final merging token counts and uniform sampling rates. Specifically, we set context sample ratio  $u = 0.1$  and source merging token  $m = 50$  for LIBERO-Spatial;  $u = 0.25, m = 80$  for Goal and Long;  $u = 0.35, m = 130$  for LIBERO-Object. The kernel size and threshold for token expanding are set to 3 and 2, respectively. All experiments are conducted on a single NVIDIA A100-40GB GPU.

### C. Main Result

Table I shows that our proposed TEAM-VLA achieves competitive results compared to other state-of-the-art methods. Without any average performance degradation, TEAM-VLA reduces the inference time of OpenVLA-OFT from 109 ms to 72.1 ms, achieving over a 1.5 $\times$  speedup. Although VLA-Cache achieves better success rate performance, its improvement on latency is marginal. When compared to EfficientVLA, with only an additional 1.5 ms of inference time, our method yields a 7.7% higher success rate. Moreover, unlike VLA-Cache and SpecPrune-VLA, our method purely

TABLE I: Experiments on the LIBERO Benchmark. Here, LM-pre denotes pruning applied before the large language model (LLM) backbone, and buffer-free indicates that no historical frame information is used for comparison.

| Method        | Success Rate (%) / Latency (ms) |             |             |             | Average<br>Success Rate $\uparrow$ | Average<br>Latency $\downarrow$ | FLOPS | LM-Pre | Buffer-Free |
|---------------|---------------------------------|-------------|-------------|-------------|------------------------------------|---------------------------------|-------|--------|-------------|
|               | spatial                         | object      | goal        | long        |                                    |                                 |       |        |             |
| OpenVLA-OFT   | 97.6 / 109                      | 96.5 / 109  | 97.9 / 109  | 94.5 / 109  | 96.6                               | 109                             | 100%  | ✓      | ✓           |
| SparseVLM     | 96.8 / 85.3                     | 94.2 / 85.3 | 97.6 / 85.3 | 93.6 / 85.3 | 95.5                               | 85.3                            | 77%   | ✓      | ✓           |
| VLA-Cache     | 99.0 / 101                      | 97.7 / 102  | 97.4 / 102  | 93.6 / 102  | <b>96.9</b>                        | 101.7                           | 83%   | ✓      | ✗           |
| EfficientVLA  | 96.5 / 68.8                     | 91.1 / 71.4 | 96.0 / 73.7 | 72.1 / 68.6 | 88.9                               | <b>70.6</b>                     | 35%   | ✗      | ✓           |
| SpecPrune-VLA | 98.2 / 72.4                     | 96.3 / 76.2 | 97.7 / 73.6 | 94.0 / 78.1 | 96.5                               | 75.1                            | 43%   | ✗      | ✗           |
| TeamVLA(Ours) | 99.2 / 68.1                     | 96.5 / 74.7 | 97.0 / 72.9 | 93.8 / 72.8 | <u>96.6</u>                        | <u>72.1</u>                     | 39%   | ✓      | ✓           |

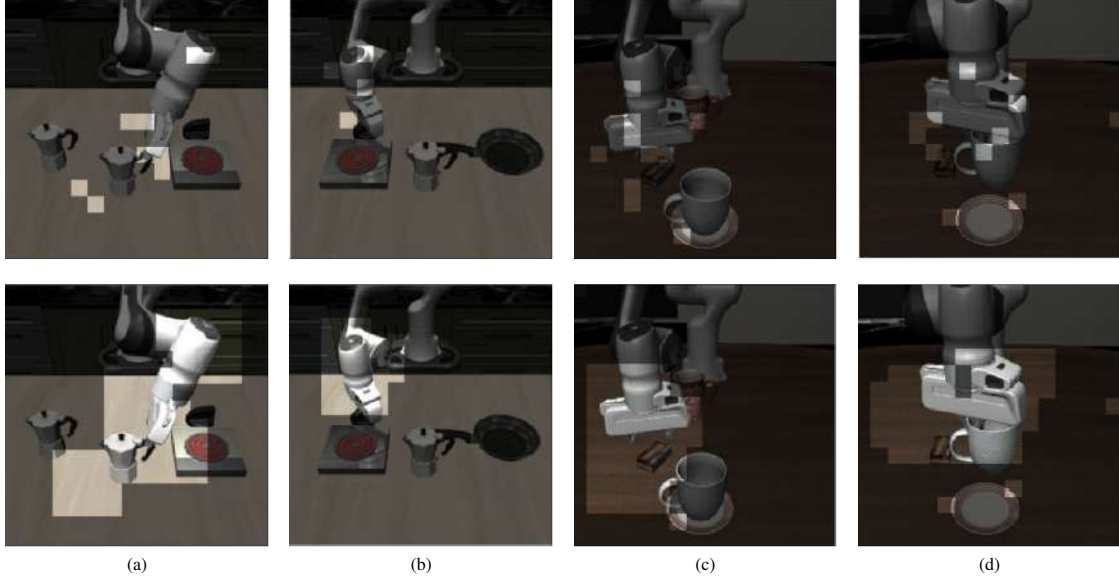


Fig. 4: Panels (a)–(d) show visualizations of different tasks in the LIBERO-10 benchmark. For each task, the top row displays the sparse similarity mask, while the bottom row presents the corresponding expanded mask.

TABLE II: Ablation Study on LIBERO-10. Pruned means the number of tokens our method pruned.  $u$  denotes the proportion of context tokens to be retained,  $K$  represents the kernel size used when computing the density map via convolution.

| Method                            | SR          | Latency     | Pruned     |
|-----------------------------------|-------------|-------------|------------|
| OpenVLA-OFT                       | 94.5        | 109         | 0          |
| + Context Sampling ( $u=0.1$ )    | 80.1        | 67.4        | 461        |
| + Context Sampling ( $u=0.25$ )   | <b>88.6</b> | <b>71.3</b> | <b>384</b> |
| + Context Sampling ( $u=0.3$ )    | 91.7        | 71.9        | 358        |
| + Context Sampling ( $u=0.4$ )    | 92.1        | 76.9        | 307        |
| + Most Similar Token              | 89.5        | 71.9        | 358        |
| + Token Expanding ( $K$ size=3)   | <b>93.8</b> | <b>76.8</b> | <b>302</b> |
| + Token Expanding ( $K$ size=5)   | 94.1        | 82.5        | 249        |
| + Action-Guided Merging (top-50)  | 88.3        | 71.6        | 462        |
| + Action-Guided Merging (top-80)  | <b>93.8</b> | <b>72.8</b> | <b>432</b> |
| + Action-Guided Merging (top-110) | 92.8        | 73.7        | 402        |
| + Action-Guided Merging (top-130) | 93.8        | 74.1        | 372        |

relies on the current observation, eliminating the need for prior knowledge derived from multiple previous frames. We also compare different pruning locations (before language model and in the middle of the language model) and examine whether historical frame information is necessary at the last two columns of Table I. For pruning location, existing

TABLE III: Ablation study on the choice of layers and Pruning vs Merging (Final top-80).

| Method   | Token SR    | Merging Latency | Token SR | Pruning Latency |
|----------|-------------|-----------------|----------|-----------------|
| Layer 1  | 72.6        | <b>68.7</b>     | 74.2     | <b>68.7</b>     |
| Layer 4  | 88.0        | 70.2            | 90.2     | 70.2            |
| Layer 8  | 90.4        | 71.5            | 92.2     | 71.5            |
| Layer 16 | <b>93.8</b> | 72.8            | 92.1     | 72.8            |

attention-based methods typically rely on the second or third layer of the language backbone, or even deeper layers, where action tokens, language tokens, and image tokens are better aligned [12], [10]. However, pruning at such depths fails to deliver meaningful speed gains due to the significant computation already incurred in earlier layers [12].

In addition, we evaluate the use of an image buffer. Previous work[8], [12], [13] identifies foreground regions by comparing patches between consecutive frames. This patch-based frame differencing method is motivated by the fact that, in robotic manipulation tasks, the moving entities are typically the robot arm or the objects being manipulated. Although such temporal differencing can quickly localize foreground regions, it introduces the need to store historical

TABLE IV: Ablation Study with Threshold  $\tau$ .

| Thres.<br>$\tau$ | Success Rate (%) / Latency (ms) / Patches Number |                          |                        |                        | Avg<br>Succ. | Avg<br>Latency | Avg<br>Patch |
|------------------|--------------------------------------------------|--------------------------|------------------------|------------------------|--------------|----------------|--------------|
|                  | spatial<br>( $u = 0.1$ )                         | object<br>( $u = 0.35$ ) | goal<br>( $u = 0.25$ ) | long<br>( $u = 0.25$ ) |              |                |              |
| $\tau = 0$       | 98.4 / 85.1 / 336                                | 97.0 / 88.5 / 375        | 97.6 / 85.4 / 323      | 95.6 / 84.9 / 335      | 97.2         | 86.0           | 342.3        |
| $\tau = 1$       | 99.2 / 69.6 / 151                                | 96.6 / 78.2 / 245        | 96.8 / 76.0 / 187      | 93.8 / 77.2 / 201      | 96.6         | 75.3           | 196.0        |
| $\tau = 2$       | 96.6 / 68.8 / 92                                 | 96.4 / 77.2 / 202        | 95.2 / 70.0 / 150      | 90.0 / 71.1 / 160      | 94.6         | 71.8           | 151.0        |

frames and relies on temporal priors, making the method less flexible. In contrast, our buffer-free strategy achieves comparable effectiveness without relying on any historical observations.

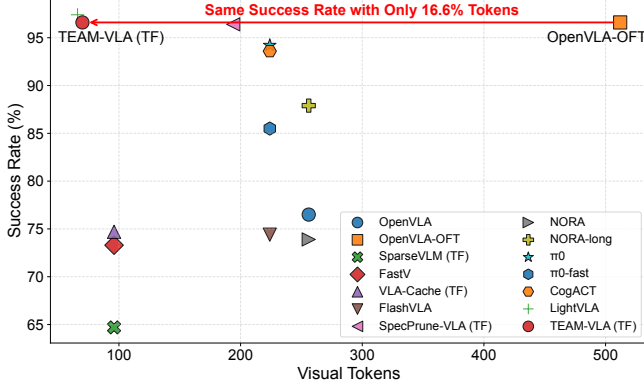


Fig. 5: We further compare the number of remaining tokens across several mainstream methods, reporting for TEAM-VLA the average token count after the merging stage. As shown, TEAM-VLA retains substantially fewer tokens than other training-free (TF) approaches while achieving significantly higher performance.

#### D. Ablation Study

1) *Ablation Study on Components*: Table II illustrates the effect of TEAM-VLA’s components. First, we evaluate the proposed Token Expanding method by comparing context sampling ( $u = 0.4$ ) with Token Expanding ( $u = 0.25, K = 3$ ). We find that, with a similar number of pruned tokens, our proposed method yields a 1.7% improvement in success rate. We also compare the performance without Token Expanding. When only language-related tokens are added, the model yields suboptimal performance with a success rate of 89.5%, which affirms the importance of Token Expanding when pruning in the first layer. We further study the effect of the kernel size in Token Expanding. With larger kernel size ( $K = 5$ ), although the success rate increases by 0.3%, the dense area expands to a larger spatial region, resulting in more visual tokens during inference and an additional 6 ms of latency.

2) *Ablation Study on Merging Layer*: In the first-stage token pruning, TEAM-VLA performs an initial pruning step after the sensory encoder processes the raw input but before the tokens are fed into the language backbone. Prior work [9] has shown that applying pruning directly at the sensory encoding stage can degrade the model’s visual representation

quality, leading to suboptimal performance. Conversely, performing pruning only within the early layers of the language backbone yields limited speedup [10], [12]. Therefore, we conduct an ablation study on the choice of layer at which merging is performed. As shown in the Token Merging column of Table III, the success rate (SR) consistently increases when merging is performed at deeper layers, which aligns with findings from prior work. Since merging serves as the second-stage acceleration in our framework, we select the middle layer, where the highest success rate is achieved, as the merging location in our experiments.

3) *Ablation Study on Threshold  $\tau$* : From the ablation results of Table IV, increasing the density threshold  $\tau$  reduces the number of expanded regions, resulting in fewer patches and lower latency. For example, the average patch count decreases from  $342 \rightarrow 196 \rightarrow 151$  when  $\tau$  increases from 0 to 1 to 2, and the corresponding average latency drops from  $86.0\text{ms} \rightarrow 75.3\text{ms} \rightarrow 71.8\text{ms}$ . However, a large  $\tau$  also reduces useful context, causing the success rate to fall from 97.2% ( $\tau = 0$ ) to 94.6% ( $\tau = 2$ ). In contrast,  $\tau = 1$  maintains high accuracy (96.6%) while significantly reducing computation. Therefore, we adopt  $\tau = 1$  as the default setting for our main experiments.

#### E. Further Analysis

1) *Merging vs Pruning*: Table III further illustrates the difference between merging and pruning. Although the performance gap is modest, merging at deeper layers consistently outperforms pruning. This is because, after the initial pruning stage, most remaining tokens already correspond to foreground or otherwise informative regions. Directly pruning these tokens down to a small budget (e.g., 80) inevitably removes valuable information. In contrast, merging aggregates less critical tokens into the most relevant ones, preserving their semantic content while reducing token count.

2) *Final Token Comparison*: The last three rows of Table II present the effect of the final number of merged tokens. When the number of tokens is reduced to 50, the success rate drops to 88.3%. However, increasing the token count to 80 leads to a substantial improvement, achieving a success rate of 93.8%. Further increasing the number of tokens yields no additional performance gains. These results demonstrate that our merging strategy effectively preserves the information carried by tokens that would otherwise be discarded. As a result, proposed action-guided merging maintains comparable success rates to the pre-merging model while simultaneously improving inference speed. We further compare the final number of retained tokens across different methods in Figure 5. Among all Training-Free (TF) approaches, our method

significantly outperforms the others, achieving substantially fewer final tokens while maintaining comparable performance. Compared to the baseline, TEAM-VLA removes the majority of tokens, on average 432, yet still preserves the same level of task success.

3) *Visualization of Expanded*: We provide a visualization of the extracted patches in Figure 4, where the first row shows the sparse image-text most similar patches, and the second row presents the patches after our expansion. As illustrated, the most similar patches contain cues about the objects and the robot arm, but their high sparsity prevents the model from capturing complete structural information about the arm or the surrounding spatial context. Our expansion method reconstructs a more coherent and spatially continuous local foreground region, thereby enabling the model to access richer visual information and ultimately leading to improved performance.

## VI. CONCLUSION

In this paper, we present TEAM-VLA, a training-free framework that efficiently identifies foreground tokens and performs action-guided merging. Unlike prior methods, TEAM-VLA requires no prior knowledge or temporal cues; instead, it uses a density-expansion strategy to reconstruct object foreground regions and the robot arm from sparse similarity signals. A mid-layer merging module further accelerates inference while preserving essential semantics. Extensive experiments show that TEAM-VLA achieves state-of-the-art performance among training-free approaches, delivering strong accuracy-efficiency trade-offs for VLA models.

## REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, *et al.*, “Open-vla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [2] K. Black, N. Brown, D. Driess, A. Esmail, *et al.*, “ $\pi 0$ : A vision-language-action flow model for general robot control. corr. abs/2410.24164, 2024. doi: 10.48550,” *arXiv preprint arXiv:2410.24164*.
- [3] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025.
- [4] Y. Ma, Z. Song, Y. Zhuang, J. Hao, and I. King, “A survey on vision-language-action models for embodied ai,” *arXiv preprint arXiv:2405.14093*, 2024.
- [5] X. Pei, Y. Chen, S. Xu, Y. Wang, Y. Shi, and C. Xu, “Action-aware dynamic pruning for efficient vision-language-action manipulation,” *arXiv preprint arXiv:2509.22093*, 2025.
- [6] Z. Yu, B. Wang, P. Zeng, H. Zhang, J. Zhang, L. Gao, J. Song, N. Sebe, and H. T. Shen, “A survey on efficient vision-language-action models,” *arXiv preprint arXiv:2510.24795*, 2025.
- [7] J. Li, K. Li, C. Gao, Y. Li, and X. Chen, “Egoprune: Efficient token pruning for egomotion video reasoning in embodied agent,” *arXiv preprint arXiv:2507.15428*, 2025.
- [8] C. Liu, J. Zhang, C. Li, Z. Zhou, S. Wu, S. Huang, and H. Duan, “Ttf-vla: Temporal token fusion via pixel-attention integration for vision-language-action models,” *arXiv preprint arXiv:2508.19257*, 2025.
- [9] T. Jiang, X. Jiang, Y. Ma, X. Wen, B. Li, *et al.*, “The better you learn, the smarter you prune: Towards efficient vision-language-action models via differentiable token pruning,” *arXiv preprint arXiv:2509.12594*, 2025.
- [10] Y. Yang, Y. Wang, Z. Wen, L. Zhongwei, C. Zou, *et al.*, “Efficientvla: Training-free acceleration and compression for vision-language-action models,” *arXiv preprint arXiv:2506.10100*, 2025.
- [11] Y. Zhang, C.-K. Fan, J. Ma, W. Zheng, T. Huang, K. Cheng, *et al.*, “Sparsevlm: Visual token sparsification for efficient vision-language model inference,” in *International Conference on Machine Learning*, 2025.
- [12] H. Wang, J. Xu, J. Pan, Y. Zhou, and G. Dai, “Specprune-vla: Accelerating vision-language-action models via action-aware self-speculative pruning,” *arXiv preprint arXiv:2509.05614*, 2025.
- [13] S. Xu, Y. Wang, C. Xia, D. Zhu, T. Huang, and C. Xu, “Vla-cache: Towards efficient vision-language-action model via adaptive token caching in robotic manipulation,” *arXiv preprint arXiv:2502.02175*, 2025.
- [14] M. Kim, S. Gao, Y.-C. Hsu, Y. Shen, and H. Jin, “Token fusion: Bridging the gap between token pruning and token merging,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 1383–1392.
- [15] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.
- [16] J. Zhang, J. Huang, S. Jin, and S. Lu, “Vision-language models for vision tasks: A survey,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 46, no. 8, pp. 5625–5644, 2024.
- [17] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, *et al.*, “Qwen technical report,” *arXiv preprint arXiv:2309.16609*, 2023.
- [18] D. Zhu, J. Chen, X. Shen, X. Li, and M. Elhoseiny, “Minigpt-4: Enhancing vision-language understanding with advanced large language models,” *arXiv preprint arXiv:2304.10592*, 2023.
- [19] Q. Li, Y. Liang, Z. Wang, L. Luo, X. Chen, Liao, *et al.*, “Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation,” *arXiv preprint arXiv:2411.19650*, 2024.
- [20] C.-Y. Hung, Q. Sun, P. Hong, A. Zadeh, C. Li, U. Tan, *et al.*, “Nora: A small open-sourced generalist vision language action model for embodied tasks,” *arXiv preprint arXiv:2504.19854*, 2025.
- [21] A. Adilkhanov, A. Yelenov, A. Seitzhanov, A. Mazhitov, A. Abdikarimov, *et al.*, “Survey on vision-language-action models,” *arXiv preprint arXiv:2502.06851*, 2025.
- [22] K. Kawaharazuka, J. Oh, J. Yamada, *et al.*, “Vision-language-action models for robotics: A review towards real-world applications,” *IEEE Access*, 2025.
- [23] A. Brohan, N. Brown, *et al.*, “Rt-1: Robotics transformer for real-world control at scale,” *arXiv preprint arXiv:2212.06817*, 2022.
- [24] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.
- [25] W. Liang, R. Zhou, Y. Ma, B. Zhang, S. Li, Y. Liao, and P. Kuang, “Large model empowered embodied ai: A survey on decision-making and embodied learning,” *arXiv preprint arXiv:2508.10399*, 2025.
- [26] L. Chen, H. Zhao, T. Liu, S. Bai, J. Lin, *et al.*, “An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models,” in *European Conference on Computer Vision*. Springer, 2024, pp. 19–35.
- [27] S. Kim, S. Shen, D. Thorsley, A. Gholami, W. Kwon, *et al.*, “Learned token pruning for transformers,” in *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, 2022, pp. 784–794.
- [28] S.-H. Lee, J. Wang, Z. Zhang, D. Fan, and X. Li, “Video token merging for long video understanding,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 13 851–13 871, 2024.
- [29] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, *et al.*, “Token merging: Your vit but faster,” *arXiv preprint arXiv:2210.09461*, 2022.
- [30] T. Ren, S. Liu, A. Zeng, J. Lin, K. Li, *et al.*, “Grounded sam: Assembling open-world models for diverse visual tasks,” 2024.
- [31] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, *et al.*, “Grounding dino: Marrying dino with grounded pre-training for open-set object detection,” *arXiv preprint arXiv:2303.05499*, 2023.
- [32] W. Ye, Q. Wu, W. Lin, and Y. Zhou, “Fit and prune: Fast and training-free visual token pruning for multi-modal large language models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 21, 2025, pp. 22 128–22 136.
- [33] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, *et al.*, “Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation,” *arXiv preprint arXiv:2508.19236*, 2025.